

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
КАФЕДРА ІНФОРМАТИКИ ТА ПРОГРАМНОЇ ІНЖЕНЕРІЇ

Курсова робота з освітнього компоненту
«Технології паралельних обчислень. Курсова робота»
Тема: Алгоритм пошуку оберненої матриці та його паралельна
реалізація мовою програмування Python

Керівник:

PhD, ст.викл. Дифучин А. Ю.

«Допущено до захисту»

«___» _____ 2024 р.

Захищено з оцінкою

Члени комісії:

Виконавець:

Орищенко Я. О.

студент групи ІП-12

залікова книжка № ІП-1224

«23» травня 2024 р.

Інна СТЕЦЕНКО

Антон ДИФУЧИН

ЗАВДАННЯ

1. Виконати огляд існуючих реалізацій алгоритму пошуку оберненої матриці, послідовних та паралельних.
2. Виконати розробку послідовного алгоритму пошуку оберненої матриці на мові програмування Python. Дослідити швидкодію алгоритму при зростанні складності обчислень та зробити висновки про необхідність паралельної реалізації алгоритму.
3. Виконати розробку паралельного алгоритму пошуку оберненої матриці на мові програмування Python. Забезпечити зручне введення даних для початку обчислень.
4. Виконати тестування алгоритму, що доводить коректність результатів обчислень.
5. Виконати дослідження швидкодії алгоритму при зростанні кількості даних для обчислень.
6. Виконати експериментальне дослідження прискорення розробленого алгоритму при зростанні кількості даних для обчислень.
7. Зробити висновки про переваги паралельної реалізації обчислень для алгоритму, що розглядається у курсовій роботі, та програмних засобів, які використовувались.

АНОТАЦІЯ

Структура та обсяг роботи. Пояснювальна записка курсової роботи складається з 5 розділів, містить 16 рисунків, 3 таблиці, 3 формули, 7 джерел.

Мета. Метою розробки є оптимізація роботи із великими обсягами даних та пришвидшення обчислень за рахунок реалізації паралельного алгоритму пошуку оберненої матриці на мові програмування Python.

Проведено дослідження існуючих послідовних та паралельних алгоритмів пошуку оберненої матриці. Реалізовано та протестовано послідовний метод пошуку оберненої матриці методом приєднаної матриці. Спроектовано та реалізовано паралельний метод пошуку матриці. Проведено дослідження ефективності паралельного алгоритму.

КЛЮЧОВІ СЛОВА: ПОШУК ОБЕРНЕНОЇ МАТРИЦІ, ПАРАЛЕЛЬНІ ОБЧИСЛЕННЯ, МЕТОД ПРИЄДНАНОЇ МАТРИЦІ.

ЗМІСТ

ВСТУП.....	5
1 ОПИС АЛГОРИТМУ ТА ЙОГО ВІДОМИХ ПАРАЛЕЛЬНИХ РЕАЛІЗАЦІЙ.....	6
1.1 Опис послідовних алгоритмів	6
1.2 Опис паралельних алгоритмів	8
2 РОЗРОБКА ПОСЛІДОВНОГО АЛГОРИТМУ ТА АНАЛІЗ ЙОГО ШВИДКОДІЇ	9
2.1 Псевдокод послідовного алгоритму	9
2.2 Реалізація послідовного алгоритму	10
2.3 Перевірка коректності роботи алгоритму	14
2.4 Тестування алгоритму	15
3 ВИБІР ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ДЛЯ РОЗРОБКИ ПАРАЛЕЛЬНИХ ОБЧИСЛЕНЬ ТА ЙОГО КОРОТКИЙ ОПИС	17
4 РОЗРОБКА ПАРАЛЕЛЬНОГО АЛГОРИТМУ З ВИКОРИСТАННЯМ ОБРАНОГО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ: ПРОЕКТУВАННЯ, РЕАЛІЗАЦІЯ, ТЕСТУВАННЯ	18
4.1 Проєктування паралельного алгоритму	18
4.2 Реалізація паралельного алгоритму	18
4.3 Перевірка коректності роботи алгоритму	21
4.4 Тестування паралельного алгоритму	23
5 ДОСЛІДЖЕННЯ ЕФЕКТИВНОСТІ ПАРАЛЕЛЬНИХ ОБЧИСЛЕНЬ АЛГОРИТМУ	24
ВИСНОВКИ	27
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	28
ДОДАТКИ	29
Додаток А. Лістинг коду.....	299

ВСТУП

У сучасному світі, в умовах неупинного розвитку технологій, постійно росте і складність задач, із якими зіштовхується людство. Виникає потреба постійної оптимізації обчислень для збільшення їх продуктивності, зменшення їх вартості та часу виконання. Одним із основних методів досягнення даних цілей є використання паралельних обчислень. Паралельні обчислення дозволяють розділити обчислювальні завдання на незалежні частини та виконувати їх одночасно на різних обчислювальних ресурсах.

Однією із важливих задач у лінійній алгебрі є пошук оберненої матриці, найбільш поширеним застосуванням якої є розв'язання систем лінійних рівнянь високих порядків, які, в свою чергу, зустрічаються при вирішенні задач у галузях фізики, економіки, інженерії та машинного навчання. При роботі із великими обсягами даних, дана задача може бути дуже обчислювально витратною, через що виникає необхідність прискорення за рахунок використання паралельних обчислень.

Одним із потужних інструментів для формалізації та моделювання таких систем є Петрі-об'єктні мережі. Цей формалізм розширює стандартний формалізм мереж Петрі, який дозволяє об'єднувати типові фрагменти мережі у окремі об'єкти, та виконувати їх тиражування.

1 ОПИС АЛГОРИТМУ ТА ЙОГО ВІДОМИХ ПАРАЛЕЛЬНИХ РЕАЛІЗАЦІЙ

Невироджена матриця[1, с. 26] – це така матриця A , визначник якої відмінний від нуля.

Обернена матриця[1, с. 26] для квадратної невиродженої матриці A – це така квадратна матриця A^{-1} , для якої виконується рівність (1.1):

$$AA^{-1} = A^{-1}A = E, \quad (1.1)$$

де E – одинична матриця того ж порядку, що матриця A та матриця A^{-1} .

Матриця є оборотною лише за умови, що вона є невиродженою.

1.1 Опис послідовних алгоритмів

Існує декілька основних алгоритмів пошуку оберненої матриці. Перший із них – метод приєднаної матриці[2, с. 25].

Приєднана матриця A^* складається із алгебраїчних доповнень кожного елементу основної матриці A , записаних у транспонованому вигляді. Для отримання оберненої матриці A^{-1} , кожен елемент матриці A^* потрібно поділити на ненульовий визначник матриці A .

Першим кроком алгоритму є знаходження визначника матриці. Якщо для матриці A визначник є нульовим, то оберненої матриці A^{-1} не існує.

Якщо визначник є ненульовим, тобто матриця A є невиродженою, то необхідно побудувати приєднану матрицю A^* за формулою (1.2):

$$A^* = (A_{ij})_{n \times n}^T = \begin{pmatrix} A_{11} & A_{12} & \dots & A_{1n} \\ A_{21} & A_{22} & \dots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \dots & A_{nn} \end{pmatrix}, \quad (1.2)$$

де $A_{ij}, i = \overline{1, n}, j = \overline{1, n}$, - алгебраїчні доповнення елементів матриці A .

Алгебраїчне доповнення[3, с. 27] A_{ij} елементу a_{ij} обчислюється як мінор M_{ij} із власним знаком, якщо сума номерів рядка i та стовпця j є парним числом, і з оберненим знаком, якщо сума є непарною. Алгебраїчні доповнення

записуються у приєднану матрицю транспоновано, тобто алгебраїчне доповнення елемента у рядку i та стовпці j буде записано у приєднану матрицю у рядок j та стовпець i .

Міnor[3, с. 28] M_{ij} для елемента a_{ij} – це визначник матриці, отриманої із матриці A , шляхом видалення із неї рядка та стовпця, на перетині яких знаходиться елемент a_{ij} .

Знайшовши приєднану матрицю A^* , обчислюємо обернену матрицю A^{-1} за формулою (1.3):

$$A^{-1} = \frac{1}{\det(A)} A^*, \quad (1.3)$$

де $\det(A)$ – визначник матриці A .

Ще один метод, який застосовується для пошуку оберненої матриці, є метод Гауса-Жордана[4]. Він полягає у створенні розширеної матриці $(A|E)$, шляхом приєднання до матриці A одиничної матриці E , після чого, за допомогою елементарних перетворень над рядками, матриця A приводиться до одиничного вигляду. При цьому, усі дії, які виконуються над рядками матриці A , виконуються і над матрицею E , і у результаті перетворення основної матриці на одиничну, приєднана одинична матриця перетвориться на обернену матрицю A^{-1} .

Спочатку створюється матриця $(A|E)$, яка має наступний вигляд:

$$\left(\begin{array}{cccc|cccc} a_{11} & a_{12} & \dots & a_{1n} & 1 & 0 & \dots & 0 \\ a_{21} & a_{22} & \dots & a_{2n} & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} & 0 & 0 & \dots & 1 \end{array} \right),$$

де n – розмірність матриці.

На кожній ітерації алгоритму виконується два основних кроки:

- 1) На першому кроці всі елементи рядка діляться на елемент основної діагоналі a_{ii} , $i = \overline{1, n}$.
- 2) На другому кроці виконуються операції над усіма елементами рядка таким чином, щоб елемент головної діагоналі a_{ii} став одиницею, а решта елементів рядка стали нулями.

Дані кроки повторюються для кожного рядка матриці A до тих пір, поки вона не отримає вигляд одиничної матриці. Аналогічні операції виконуються і над приєднаною справа одиничною матрицею. Після завершення роботи алгоритму, матриця справа і буде оберненою матрицею A^{-1} для початкової матриці A :

$$\left(\begin{array}{cccc|cccc} 1 & 0 & \dots & 0 & a'_{11} & a'_{12} & \dots & a'_{1n} \\ 0 & 1 & \dots & 0 & a'_{21} & a'_{22} & \dots & a'_{2n} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 & a'_{n1} & a'_{n2} & \dots & a'_{nn} \end{array} \right),$$

де $a'_{ij}, i = \overline{1, n}, j = \overline{1, n}$, – елементи оберненої матриці A^{-1} .

1.2 Опис паралельних алгоритмів

Головною умовою існування паралельної реалізації алгоритму є існування у послідовному алгоритмі дій, які є незалежними одна від одної, а отже які можна виконувати одночасно, використовуючи різні обчислювальні ресурси.

У паралельній реалізації методу приєднаної матриці такими незалежними діями є пошук алгебраїчних доповнень для елементів матриці. Кожне алгебраїчне доповнення може бути обчислено незалежно від інших, та записано у відповідне місце результуючої матриці.

У паралельній реалізації методу Гауса-Жордана[4], хоча дії на кожній ітерації і є залежними одна від одної, проте операції всередині однієї ітерації можна виконувати паралельно. Таким чином, матриця розділяється між обчислювальними ресурсами на рядки, які зводять усі елементи рядка, окрім елемента a_{ii} до нульових значень, шляхом виконання елементарних перетворень над рядками.

2 РОЗРОБКА ПОСЛІДОВНОГО АЛГОРИТМУ ТА АНАЛІЗ ЙОГО ШВИДКОДІЇ

Для реалізації послідовного алгоритму пошуку оберненої матриці на мові програмування Python, було обрано метод приєднаної матриці.

2.1. Псевдокод послідовного алгоритму

1. ПОЧАТОК

2. Перевірити умови існування оберненої матриці

2.1. **ЯКЩО** кількість рядків матриці не дорівнює кількості стовпців матриці, **ТО** перейти до пункту 4.

2.2. **Обчислити** визначник основної матриці.

2.3. **ЯКЩО** визначник матриці рівний нулю, **ТО** перейти до пункту 4.

3. ЦИКЛ проходження по усім рядкам матриці:

3.1. ЦИКЛ проходження по усім елементам у рядку:

3.1.1. Отримати значення мінору елемента.

3.1.1.1. **Отримати** матрицю, яка є основною матрицею, у якій були вилучені рядок та стовпчик, у яких знаходиться елемент, мінор якого обчислюється.

3.1.1.2. **Обчислити** визначник отриманої матриці.

3.1.2. Отримати значення алгебраїчного доповнення елемента.

3.1.2.1. **Обчислити** алгебраїчне доповнення елемента, поділивши значення мінору елемента на визначник основної матриці.

3.1.2.2. **ЯКЩО** сума номеру рядка та номеру стовпця поточного елемента не є парним числом, **ТО** змінити знак значення алгебраїчного доповнення.

3.1.3. **Транспоновано** записати отримане значення алгебраїчного доповнення у результуючу матрицю.

4. КІНЕЦЬ

2.2. Реалізація послідовного алгоритму

Для реалізації послідовного алгоритму на мові програмування Python, створимо клас `Matrix`, у якому зберігатиметься об'єкт матриці, а також будуть реалізовані усі необхідні методи для обчислення оберненої матриці.

Матриця зберігатиметься у вигляді масиву бібліотеки `NumPy`[5], замість списків Python. Це дозволить значно виграти у швидкодії алгоритму завдяки використанню методів `NumPy` для обробки матриць.

На рисунку 2.1 зображено конструктор класу `Matrix`. Користувач повинен задати кількість рядків та стовпців у матриці, а також, за бажанням, може одразу ініціалізувати матрицю.

```
class Matrix:
    def __init__(self, rows: int, cols: int, matrix=None):
        self.rows = rows
        self.cols = cols

        if matrix is not None:
            self.set_matrix(matrix)
        else:
            self.matrix = np.zeros(shape=(rows, cols), dtype=np.float32)
```

Рисунок 2.1 – конструктор класу `Matrix`

Користувач може ініціалізувати матрицю за допомогою трьох методів. Перший метод – `set_matrix()` – дозволяє користувачеві власноруч створити матрицю та записати її у об'єкт класу `Matrix`. Другий метод – `generate_random_matrix()` – генерує кожне значення матриці як випадкове дійсне число у заданих користувачем межах. Третій метод – `generate_test_matrix()` – ініціалізує матрицю як одиничну, тобто усі елементи основної діагоналі є одиницями, а решта елементів є нулями. Методи ініціалізації зображені на рисунку 2.2.

```
def set_matrix(self, matrix):
    self.matrix = np.array(matrix, dtype=np.float32)

def generate_random_matrix(self, min_value: float, max_value: float):
    self.matrix = np.random.uniform(low=min_value, high=max_value + 1, size=(self.rows, self.cols))

def generate_test_matrix(self):
    self.matrix = np.eye(self.rows, self.cols, dtype=np.float32)
```

Рисунок 2.2 – методи ініціалізації матриці

Було створено методи для отримання та встановлення параметрів матриці:

- `get_value()` – отримання значення елемента за номером рядка та стовпця;
- `set_value()` – встановлення значення елемента матриці за номером рядка та стовпця;
- `get_rows()` – отримати кількість рядків у матриці;
- `get_cols()` – отримати кількість стовпчиків у матриці;
- `pprint()` – вивести вигляд матриці у консоль.

Код даних методів зображено на рисунку 2.3:

```
def get_value(self, row: int, col: int):
    return self.matrix[row, col]

def set_value(self, row: int, col: int, value: float):
    self.matrix[row, col] = value

def get_rows(self) -> int:
    return self.rows

def get_cols(self) -> int:
    return self.cols

def pprint(self):
    print(self.matrix)
```

Рисунок 2.3 – методи отримання та встановлення параметрів матриці

Перейдемо до методу для послідовного обчислення оберненої матриці за допомогою алгоритму приєднаної матриці. Створимо метод `inverse_matrix()`,

який обчислює послідовну матрицю, та повертає її у вигляді об'єкту класу Matrix. Код методу для отримання оберненої матриці за допомогою алгоритму оберненої матриці зображено на рисунку 2.4:

```
# послідовний алгоритм пошуку оберненої матриці
def inverse_matrix(self):
    # перевірка можливості обернення матриці
    if self.get_rows() != self.get_cols():
        raise Exception('Matrix is not square, impossible to calculate inverse matrix')

    det_sign, log_det = self.slog_determinant()
    if np.round(np.exp(log_det)) == 0:
        raise Exception('Matrix determinant is 0, impossible to calculate inverse matrix')

    inverse_matrix = Matrix(self.rows, self.cols)

    start = time.time()

    for row in range(self.rows):
        for col in range(self.cols):
            # алгебраїчне доповнення елемента matrix[row][col]
            minor_sign, log_minor = self.get_matrix_minor(row, col)

            value = np.exp(log_minor - log_det) * minor_sign * det_sign

            # якщо сума індексів непарна - змінюємо знак
            if (row + col) % 2 != 0:
                value = -value

            # транспоновано встановлюємо значення оберненої матриці
            inverse_matrix.set_value(col, row, value)

    end = time.time()
    print(f'Sequential inversion time: {end - start}')

    return inverse_matrix
```

Рисунок 2.4 – метод для отримання оберненої матриці

Згідно описаному у псевдокодї алгоритму, даний код:

- 1) Перевіряє, чи є матриця квадратною;
- 2) Знаходить знак визначника та значення натурального логарифму визначника основної матриці;
- 3) Перевіряє, чи є матриця невиродженою, обчисливши значення визначника матриці, та порівнявши його із нулем;
- 4) Для кожного елемента матриці, виконує пошук мінора за допомогою метода get_matrix_minor();

- 5) Отримавши знак мінора та натуральний логарифм мінора, обчислює значення алгебраїчного доповнення, підносячи число e у степінь, яка є різницею натурального логарифма мінора і натурального логарифма визначника основної матриці, та перемножуючи знаки визначника матриці та мінора;
- 6) Змінює знак алгебраїчного доповнення, якщо сума номеру рядка та стовпця елементу є непарним числом;
- 7) Транспоновано записує отримане алгебраїчне доповнення у результуючу матрицю;
- 8) Заміряє час пошуку оберненої матриці.

Отримання визначника матриці виконується за допомогою метода `slog_determinant()` (рисунок 2.5). За допомогою метода `np.linalg.slogdet()` із бібліотеки NumPy, цей метод повертає знак визначника переданої йому матриці, та натуральний логарифм визначника. Було використано саме такий підхід до обчислення визначника тому, що при його обчисленні для великих матриць (розмірністю понад 100x100 елементів), значення визначника є дуже великим, через що методи, які вираховують його напрямку, наприклад `np.linalg.det()` із бібліотеки NumPy, або `tf.linalg.det()` із бібліотеки TensorFlow[6], округлюють його до нескінченності.

```
# функція для отримання знаку визначника матриці та натурального логарифма визначника
def slog_determinant(self) -> tuple[float, float]:
    sign, log_det = np.linalg.slogdet(self.matrix)
    return sign, log_det
```

Рисунок 2.5 – метод для обчислення визначника матриці

Міnor матриці обчислюється за допомогою методу `get_matrix_minor()` (рисунок 2.6). Тут, за допомогою методів бібліотеки NumPy, із матриці видаляються необхідні рядок та стовпчик. Після цього, за допомогою метода `tf.linalg.slogdet()` бібліотеки TensorFlow, обчислюється знак визначника матриці та натуральний логарифм визначника.

```
# отримання мінора матриці - визначник основної матриці, у якій видалені задані рядок та стовпець
# результат повертається як знак визначника та значення натурального логарифма визначника
def get_matrix_minor(self, row_to_delete: int, col_to_delete: int) -> tuple[float, float]:
    # Отримання матриці без заданого рядка та стовпця
    m = np.delete(np.delete(self.matrix, row_to_delete, axis=0), col_to_delete, axis=1)

    # обчислення визначника матриці
    sign, logdet = tf.linalg.slogdet(m)

    return sign.numpy(), logdet.numpy()
```

Рисунок 2.6 – метод для обчислення мінору елемента матриці

2.3. Перевірка коректності роботи алгоритму

Згідно формулі (1.1), обернена матриця A^{-1} вважається оберненою до матриці A , якщо у результаті множення цих двох матриць отримується одинична матриця E такої ж розмірності, як і матриці A та A^{-1} .

Перевіримо роботу алгоритму пошуку оберненої матриці методом приєднаної матриці, створивши метод `single_sequential_test()`, у якому:

- 1) Створимо матрицю та ініціалізуємо її випадковими значеннями;
- 2) Виконаємо пошук оберненої матриці;
- 3) Виконаємо множення початкової матриці та отриманої оберненої матриці за допомогою метода `np.matmul()` бібліотеки NumPy.

Якщо результатом множення буде одинична матриця, то розроблений алгоритм пошуку оберненої матриці працює правильно.

На рисунку 2.7 можна побачити код для тестування коректності роботи алгоритму.

```
def single_sequential_test():
    size = 5
    m = Matrix(size, size)
    m.generate_random_matrix(-100, max_value=100)
    print('\nMatrix:')
    m.pprint()

    inverse = m.inverse_matrix()
    print('\nInverse matrix:')
    inverse.pprint()

    print('\nInitial matrix and inverse matrix multiplication:')
    print(np.round(np.matmul(inverse.matrix, m.matrix)))
```

Рисунок 2.7 – метод для тестування послідовного алгоритму

Виконаємо дану функцію, та перевіримо результати її роботи (рисунок 2.8).

```
Matrix:
[[-21.71430733  54.51762931  30.63037349  -4.99986371  23.03682061]
 [ 39.29840875  73.712395   93.84554435  90.59078984  78.11910008]
 [ 63.65586841   2.7325151  28.03547453   5.25012118  32.70965553]
 [ 32.2793796  -43.47382682  15.31529338  50.10583573   6.18013432]
 [ 13.11296089 -50.04320625  99.62801941  52.74354825   3.36315455]]
Sequential inversion time: 0.02399158477783203

Inverse matrix:
[[-0.07334519  0.02416978  0.00439858 -0.05958943  0.00770546]
 [-0.07625529  0.03239879 -0.01069752 -0.07252292  0.00708501]
 [-0.01590183  0.00554136  0.00145941 -0.02570284  0.01324705]
 [-0.03481214  0.01792198 -0.01553994 -0.01478137  0.00046634]
 [ 0.1683236  -0.05736904  0.02414902  0.14642712 -0.02701628]]

Initial matrix and inverse matrix multiplication:
[[ 1.  0.  0.  0.  0.]
 [ 0.  1. -0.  0. -0.]
 [-0.  0.  1. -0. -0.]
 [ 0. -0. -0.  1. -0.]
 [-0.  0.  0.  0.  1.]]
```

Рисунок 2.8 – результат тестування алгоритму

Як можна побачити, результатом множення початкової матриці є одинична матриця, що доводить коректність роботи послідовного алгоритму пошуку оберненої матриці методом приєднаної матриці.

2.4. Тестування алгоритму

Знаючи, що алгоритм працює коректно, можна перейти до тестування швидкодії алгоритму на матрицях великих розмірностей. Проведемо пошук оберненої матриці для матриць розмірністю від 100 до 450 із кроком 50. Результат тестування наведено у таблиці 2.1.

Таблиця 2.1 – тестування послідовного алгоритму

Розмір матриці	Час виконання послідовним алгоритмом, с
100	1,606
150	6,903

200	22,008
250	53,728
300	149,212
350	271,181
400	662,048
450	1183,914

Візуалізацію залежності часу пошуку оберненої матриці від розмірності матриці представлено на рисунку 2.9.



Рисунок 2.9 – візуалізація результатів тестування послідовного алгоритму

З ВИБІР ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ДЛЯ РОЗРОБКИ ПАРАЛЕЛЬНИХ ОБЧИСЛЕНЬ ТА ЙОГО КОРОТКИЙ ОПИС

Для виконання завдання курсової роботи було обрано мову програмування Python. У більшості мов програмування, основним підходом до використання паралельних обчислень, є використання багатопоточності. Як і інші мови, Python також підтримує таку технологію, проте ключовою відмінністю є те, що ця мова, у своїй еталонній реалізації із інтерпретатором CPython, використовує глобальне блокування інтерпретатора (GIL), яке у окремий момент часу дозволяє роботу лише одного потоку. Таким чином, використовуючи потоки у Python, неможливо отримати прискорення, розділивши між ними виконання незалежних частин алгоритму.

Рішенням до цієї проблеми є використання процесів та модуля Multiprocessing[7]. Фактично, процес – це окрема програма із виділеною під неї областю пам'яті та власним GIL. Процеси виконуються незалежно одне від одного, і завдяки ним можна досягти справжньої паралельності.

Для розробки паралельного алгоритму було обрано механізм пулу процесів. Пул процесів дозволяє керувати групою процесів для виконання незалежних задач. При ініціалізації пулу, створюється задана кількість процесів, які після цього беруть на виконання задачі із черги, що були завантажені у пул, виконують їх та повертають результат.

4 РОЗРОБКА ПАРАЛЕЛЬНОГО АЛГОРИТМУ З ВИКОРИСТАННЯМ ОБРАНОГО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ: ПРОЄКТУВАННЯ, РЕАЛІЗАЦІЯ, ТЕСТУВАННЯ

4.1. Проєктування паралельного алгоритму

Згідно опису паралельної реалізації алгоритму пошуку оберненої матриці методом приєднаної матриці у пункті 1.2, пошук мінорів матриці та наступне обчислення алгебраїчних доповнень є повністю незалежними процесами, які можна виконувати паралельно.

Оскільки було прийнято рішення виконувати реалізацію паралельного алгоритму із використанням пулу процесів, то пошук оберненої матриці необхідно розбити на задачі, які завантажуватимуться у чергу пулу. Однією такою задачею буде обчислення мінорів та алгебраїчних доповнень одного ряду основної матриці. Результатом виконання такої задачі буде один стовпчик оберненої матриці. Після завершення виконання задач та отримання усіх результатів, стовпчики будуть об'єднані у одну матрицю, яка і буде оберненою до початкової матриці.

4.2. Реалізація паралельного алгоритму

Для реалізації паралельного алгоритму пошуку оберненої матриці методом приєднаної матриці, у класі `Matrix` було створено метод `inverse_matrix_process_pool()`. Даний метод виконує пошук оберненої матриці та повертає її у вигляді об'єкта класу `Matrix`. Код методу можна побачити на рисунку 4.1.

```

# алгоритм пошуку оберненої матриці із використанням пулу процесів
def inverse_matrix_process_pool(self, processes_amount: int):
    # перевірка можливості обернення матриці
    if self.get_rows() != self.get_cols():
        raise Exception('Matrix is not square, impossible to calculate inverse matrix')

    det_sign, log_det = self.slog_determinant()
    if np.round(np.exp(log_det)) == 0:
        raise Exception('Matrix determinant is 0, impossible to calculate inverse matrix')

    # створення пулу процесів та списку для зберігання результатів роботи функцій
    pool = Pool(processes=processes_amount)
    futures = []

    start = time.time()

    # додавання задач у пул процесів
    for row in range(self.rows):
        futures.append(pool.apply_async(self._inverse_matrix_worker, args=(row, det_sign, log_det)))

    pool.close()

    # очікування кінця виконання задач у пулі процесів
    for future in futures:
        future.wait()

    # об'єднання результатів виконання задач в одну матрицю
    inverse_matrix = np.column_stack([future.get() for future in futures])

    end = time.time()
    print(f'Process pool inversion time: {end - start}')

    return Matrix(self.rows, self.cols, inverse_matrix)

```

Рисунок 4.1 – метод `inverse_matrix_process_pool()` для паралельного пошуку оберненої матриці

Даний код:

- 1) Перевіряє, чи є матриця квадратною;
- 2) Знаходить знак визначника `det_sign` та значення натурального логарифму визначника основної матриці `log_det` за допомогою метода `slog_determinant()`;
- 3) Перевіряє, чи є матриця невиродженою, обчисливши значення визначника матриці за допомогою `np.round(np.exp(log_det))`, та порівнявши його із нулем;
- 4) Створює пул `pool` із заданою кількістю процесів `processes_amount`;
- 5) Створює список `futures`, у якому зберігаються результати виконання задач;

- 6) За допомогою метода пулу процесів `apply_async`, завантажує асинхронні задачі у пул. Задачею виступає метод `_inverse_matrix_worker()`, у який передаються індекс рядка `row`, знак визначника матриці `det_sign` та натуральний логарифм визначника матриці `log_det`. У пул завантажується стільки задач, скільки рядків має матриця `matrix`.
- 7) Виконує закриття пула процесів за допомогою метода `pool.close()`, щоб пул не очікував отримання нових задач;
- 8) Очікує завершення усіх задач за допомогою `future.wait()`;
- 9) Отримує результат виконання усіх задач – стовпчики оберненої матриці, за допомогою `future.get()`, та об'єднує усі стовпчики у єдину матрицю за допомогою метода `np.column_stack()`;
- 10) Заміряє час виконання паралельного алгоритму.

Основна робота алгоритму виконується саме у задачах, які завантажуються у пул процесів. Кожна задача обчислює один стовпчик оберненої матриці. Код метода `_inverse_matrix_worker()`, який є задачею для пула процесів, наведено на рисунку 4.2.

```
# задача для процесів пошуку оберненої матриці
def _inverse_matrix_worker(self, row: int, det_sign: float, log_det: float) -> np.ndarray:
    result = np.zeros(shape=(self.rows, 1), dtype=np.float32)

    for col in range(self.cols):
        # алгебраїчне доповнення елементу matrix[row][col]
        minor_sign, log_minor = self.get_matrix_minor(row, col)

        value = np.exp(log_minor - log_det) * minor_sign * det_sign

        # якщо сума індексів непарна - міняємо знак
        if (row + col) % 2 != 0:
            value = -value

        # транспоновано встановлюємо значення оберненої матриці
        result[col, 0] = value

    return result
```

Рисунок 4.2 – метод `_inverse_matrix_worker()` для обчислення одного стовпчика оберненої матриці

Цей код виконує наступні дії:

- 1) Створює масив `result` – одновірний масив, який має таку ж довжину, як і розмірність матриці. У нього будуть записуватись обчислені алгебраїчні доповнення.
- 2) Для кожного елементу рядка, індекс якого було передано даному методу, виконує обчислення мінора. За допомогою метода `get_matrix_minor()`, у змінну `minor_sign` записується знак мінора, а у змінну `log_minor` – значення натурального логарифму мінора.
- 3) Обчислює значення алгебраїчного доповнення, підносячи число e у степінь, яка є різницею натурального логарифма мінора і натурального логарифма визначника основної матриці, та перемножуючи знаки визначника матриці та мінора. Отримане значення присвоює змінній `value`;
- 4) Змінює знак алгебраїчного доповнення, якщо сума номеру рядка та стовпця даного елемента є непарним числом;
- 5) Записує обчислене значення алгебраїчного доповнення у масив `result`;
- 6) Повертає масив `result` – стовпець оберненої матриці.

4.3. Перевірка коректності роботи алгоритму

Застосовуючи формулу (1.1), перевіримо коректність паралельного алгоритму, виконавши множення початкової матриці, та отриманої оберненої матриці у результаті виклику метода `inverse_matrix_process_pool()`. Якщо алгоритм працює коректно, то результатом множення буде одинична матриця такої ж розмірності, що і початкова матриця.

Створимо функцію `single_parallel_test()`, у якій створимо матрицю розмірністю 6×6 та ініціалізуємо її випадковими числами від -100 до 100. Отримавши обернену матрицю, викликавши метод `inverse_matrix_process_pool()` для двох потоків, за допомогою метода `np.matmul()` бібліотеки NumPy виконаємо множення початкової матриці та оберненої. Код функції наведено на рисунку 4.3.

```
def single_parallel_test():
    size = 5
    m = Matrix(size, size)
    m.generate_random_matrix(-100, max_value: 100)
    print('\nMatrix:')
    m.pprint()

    inverse = m.inverse_matrix_process_pool(2)
    print('\nInverse matrix:')
    inverse.pprint()

    print('\nInitial matrix and parallel inverse matrix multiplication:')
    print(np.round(np.matmul(inverse.matrix, m.matrix)))
```

Рисунок 4.3 – функція single_parallel_test() для перевірки коректності роботи паралельного алгоритму

Результат виконання функції single_parallel_test() наведено на рисунку 4.3.

```
Matrix:
[[ 95.52027756 -55.37908647 -33.92880956 -63.11587683  3.07714672]
 [ 22.47064876  59.49720204 -3.05233366 -8.13619395 11.02339629]
 [ 22.99984246  62.6355899  31.82817652 -88.77542262 -59.9226828 ]
 [ 43.15589096 -56.71695453  56.4537986  -46.79895482 11.34278731]
 [-2.53162033  43.36981979 -58.11118585 -56.70153643 -42.01444234]]
Process pool inversion time: 3.219155788421631

Inverse matrix:
[[ 0.01536746 -0.0002471  0.01147808 -0.01801414 -0.02017315]
 [-0.00215157  0.01120294  0.00179939 -0.00204978 -0.000338 ]
 [-0.00284746 -0.00061795  0.00680154  0.00244734 -0.00941059]
 [ 0.01019942 -0.00869325  0.01100975 -0.0253286  -0.02407445]
 [-0.01297343  0.02416611 -0.02310001  0.02976736  0.02257155]]

Initial matrix and parallel inverse matrix multiplication:
[[ 1.  0. -0.  0.  0.]
 [-0.  1.  0.  0. -0.]
 [ 0.  0.  1. -0. -0.]
 [-0.  0.  0.  1.  0.]
 [-0.  0. -0.  0.  1.]]
```

Рисунок 4.4 – результат тестування паралельного алгоритму

Як можна побачити, у результаті множення початкової матриці та оберненої матриці, отриманої у результаті виконання паралельного алгоритму, було отримано одиничну матрицю. Отже, можна зробити висновок, що паралельний алгоритм пошуку оберненої матриці методом приєднаної матриці працює коректно.

4.4. Тестування паралельного алгоритму

Виконаємо тестування швидкодії паралельного алгоритму пошуку оберненої матриці із зростанням розмірності матриці. Виконаємо тестування для розмірностей матриць від 100x100 до 450x450 із кроком 50. Для тестування використовуватимемо шість процесів. Результати тестування представлено у таблиці 4.1.

Таблиця 4.1 – тестування паралельного алгоритму

Розмір матриці	Час виконання паралельним алгоритмом, с
100	4,932
150	6,107
200	13,908
250	30,128
300	67,032
350	138,132
400	290,449
450	497,858

Візуалізацію залежності часу пошуку оберненої матриці від розмірності матриці представлено на рисунку 2.9.



Рисунок 4.5 – візуалізація результатів тестування послідовного алгоритму

5 ДОСЛІДЖЕННЯ ЕФЕКТИВНОСТІ ПАРАЛЕЛЬНИХ ОБЧИСЛЕНЬ АЛГОРИТМУ

Проведемо дослідження ефективності паралельного алгоритму, провівши тестування на матрицях розміром від 100 до 450 із кроком 50, елементами яких є дійсні числа від -100 до 100. Виконаємо тестування паралельного алгоритму для 2, 4, 6, 8, 12 та 16 процесів, а також для послідовного алгоритму, щоб у результаті виконати їх порівняння та обчислити прискорення, яке було отримане за допомогою паралельного алгоритму. Результати тестування представлені у таблиці 5.1.

Таблиця 5.1 – тестування паралельного алгоритму

Розмір матриці		100	150	200	250	300	350	400	450
Послідовний алгоритм		1,606	6,903	22,008	53,728	149,212	271,181	662,048	1183,914
2 процеси	Час, с	4,060	6,901	15,128	35,714	94,916	179,185	399,369	717,362
	Прискорення	0,396	1,000	1,455	1,504	1,572	1,513	1,658	1,650
4 процеси	Час, с	4,099	5,658	14,742	29,119	69,687	153,138	307,986	501,642
	Прискорення	0,392	1,220	1,493	1,845	2,141	1,771	2,150	2,360
6 процесів	Час, с	4,932	6,107	13,908	30,128	67,032	138,132	290,449	497,858
	Прискорення	0,326	1,130	1,582	1,783	2,226	1,963	2,279	2,378
8 процесів	Час, с	5,810	7,421	14,776	30,461	69,179	141,380	307,953	546,150
	Прискорення	0,276	0,930	1,489	1,764	2,157	1,918	2,150	2,168
12 процесів	Час, с	7,645	9,626	17,573	35,224	75,314	158,187	340,221	605,930
	Прискорення	0,210	0,717	1,252	1,525	1,981	1,714	1,946	1,954
16 процесів	Час, с	8,839	11,885	19,031	36,853	76,503	168,192	343,192	608,238
	Прискорення	0,182	0,581	1,156	1,458	1,950	1,612	1,929	1,946

Візуалізація залежності часу виконання алгоритму від розмірності матриці зображена на рисунку 5.1, а візуалізація отриманого прискорення у порівнянні із послідовним алгоритмом зображена на рисунку 5.2.

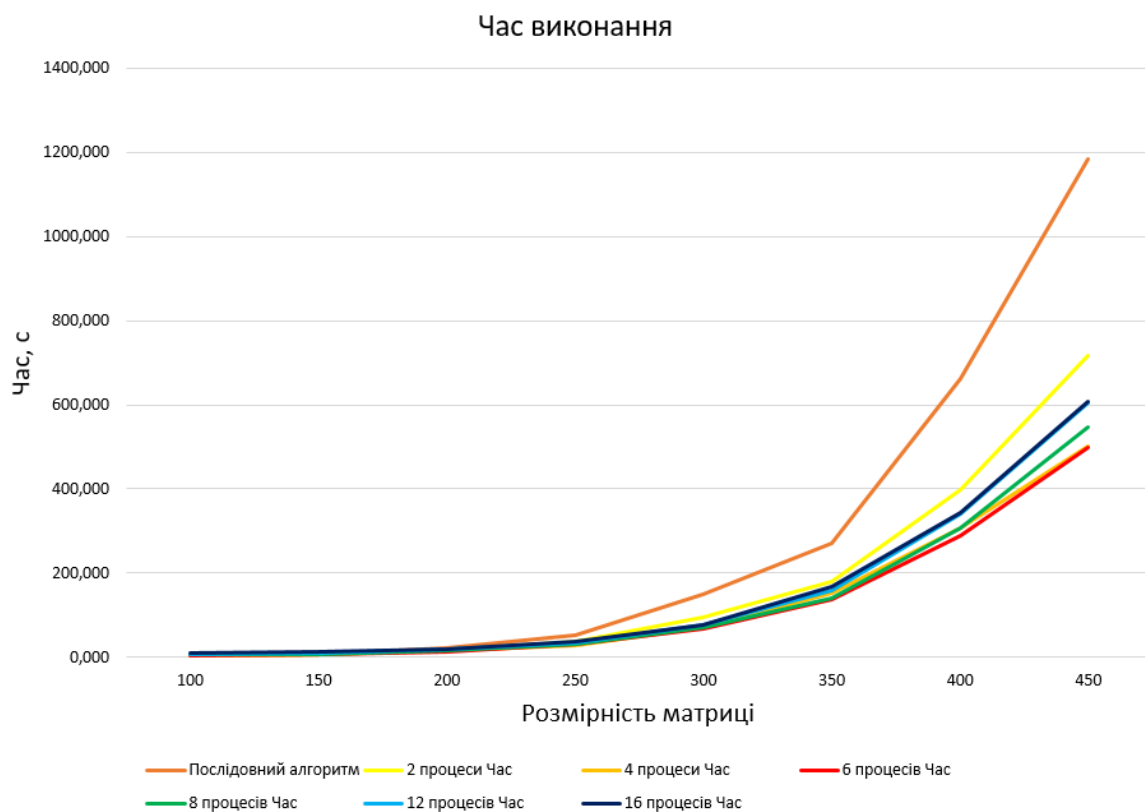


Рисунок 5.1 – візуалізація часу виконання паралельних алгоритмів

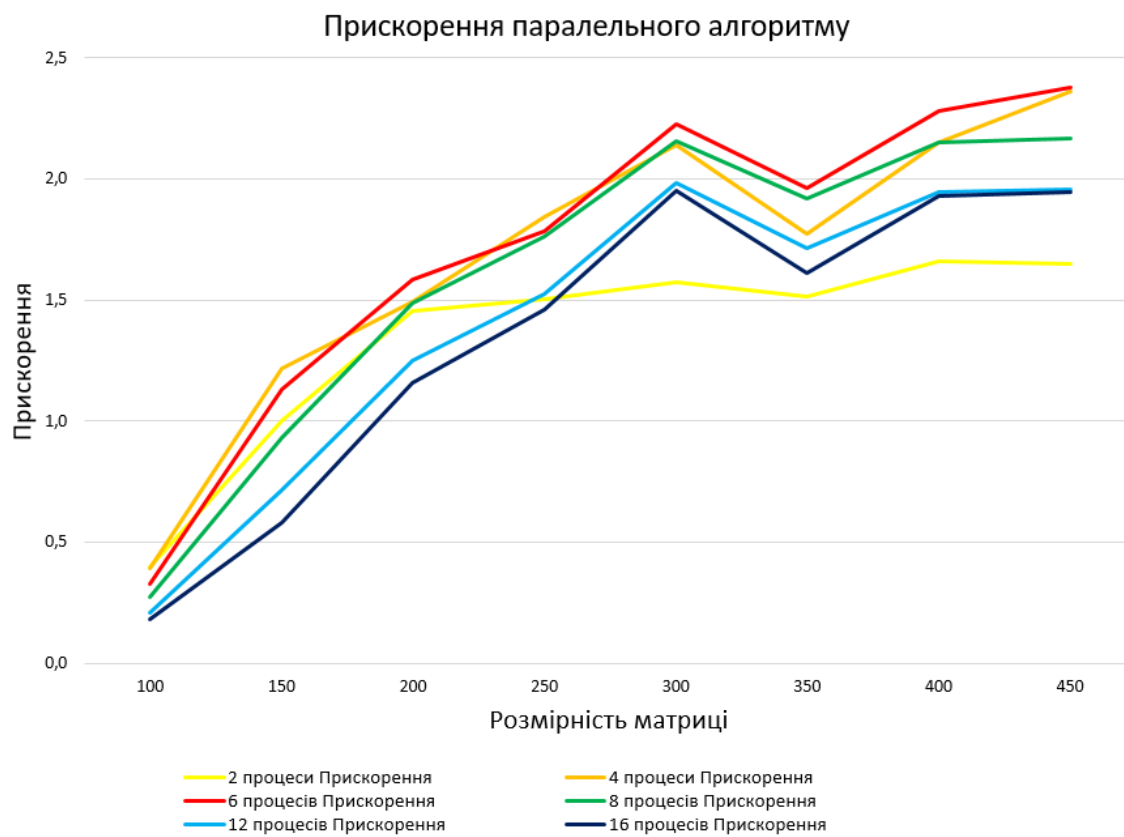


Рисунок 5.2 – візуалізація прискорення паралельних алгоритмів

Як можна побачити, паралельний алгоритм збільшує свою ефективність із збільшенням об'ємів даних та дозволяє отримати прискорення у понад 2 рази у порівнянні із послідовним алгоритмом. У результаті, найкраще прискорення, що було отримано, становить 2.378 разів, при використанні 6 процесів та при розмірності матриці 450x450.

У ході тестування було виявлено, що у більшості випадків найкраще прискорення давав алгоритм із шістьма процесами, оскільки саме стільки фізичних ядер має апаратне забезпечення, на якому проводилось тестування.

Можна дійти виснову, що при розмірностях матриць понад 150x150, варто використовувати паралельний алгоритм для пошуку оберненої матриці методом приєднаної матриці, оскільки це дозволяє значно виграти у швидкодії. Найбільш ефективним буде використання такої кількості процесів, яка рівна або близька кількості фізичних ядер апаратного забезпечення.

ВИСНОВКИ

У ході виконання курсової роботи було проведено дослідження алгоритмів для знаходження оберненої матриці. Було наведено детальний опис послідовних та паралельних реалізацій двох методів: методу приєднаної матриці та методу Гауса-Жордана. Для реалізації було обрано метод приєднаної матриці.

Для реалізації паралельного алгоритму, було використано мову програмування Python. Через існування механізму глобального блокування інтерпретатора у стандартному інтерпретаторі CPython, замість потоків було прийнято рішення використовувати процеси та модуль Multiprocessing. Для асинхронного виконання задач було вирішено застосовувати пул процесів.

Було реалізовано паралельний алгоритм пошуку оберненої матриці методом приєднаної матриці. У ході тестування було доведено ефективність алгоритму для матриць розміром понад 150×150 . Паралельний алгоритм збільшує свою ефективність із збільшенням розмірності матриці, що дозволило отримати прискорення у 2.378 разів при використанні шести процесів для матриці розміром 450×450 . Найкращу ефективність алгоритму було досягнуто при кількості процесів, яка рівна кількості фізичних ядер апаратного забезпечення, на якому проводяться розрахунки.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Лінійна алгебра та аналітична геометрія / Ю. К. Рудавський, П. П. Костробій, Х. П. Луник, Д. В. Уханська. Львів : Бескид Біт, 2002. URL: https://pdf.lib.vntu.edu.ua/books/Rudavskiy_2002_260.pdf.
2. Лінійна алгебра та аналітична геометрія / І. В. Булдігін, І. В. Алексєєва, В. О. Гайдей, О. О. Диховичний, Н. Р. Коновалова, Л. Б. Федорова та ін. Київ, 2011. URL: <https://matan.kpi.ua/public/files/Posibnyk%20LA+AG.pdf>.
3. Літнарів Р. М. Лінійна алгебра. Елементи теорії визначників. Рівне, 2006. URL: <https://core.ac.uk/download/pdf/14034614.pdf>.
4. C. Vancea, F. Vancea. Parallel Algorithm for Computing Matrix Inverse by Gauss-Jordan Method. Orladea. URL: https://www.researchgate.net/publication/38112225_Parallel_Algorithm_for_Computing_Matrix_Inverse_by_Gauss-Jordan_Method/fulltext/0ffc5d310cf255165fc80893/Parallel-Algorithm-for-Computing-Matrix-Inverse-by-Gauss-Jordan-Method.pdf.
5. NumPy. Version 1.26. URL: <https://numpy.org/doc/stable/reference/index.html>.
6. TensorFlow. Version 2.16.1. URL: https://www.tensorflow.org/api_docs.
7. Multiprocessing. URL: <https://docs.python.org/3/library/multiprocessing.html>.

ДОДАТКИ

Додаток А. Лістинг коду

Файл **matrix.py**

```
import os
import time
from typing import Tuple, Any

import numpy as np
from multiprocessing import Pool

os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'

import tensorflow as tf

class Matrix:
    def __init__(self, rows: int, cols: int, matrix=None):
        self.rows = rows
        self.cols = cols

        if matrix is not None:
            self.set_matrix(matrix)
        else:
            self.matrix = np.zeros((rows, cols), dtype=np.float32)

    def set_matrix(self, matrix):
        self.matrix = np.array(matrix, dtype=np.float32)

    def generate_random_matrix(self, min_value: float, max_value: float):
```

```

        self.matrix = np.random.uniform(low=min_value, high=max_value + 1,
size=(self.rows, self.cols))

```

```

def generate_test_matrix(self):

```

```

    self.matrix = np.eye(self.rows, self.cols, dtype=np.float32)

```

```

def get_value(self, row: int, col: int):

```

```

    return self.matrix[row, col]

```

```

def set_value(self, row: int, col: int, value: float):

```

```

    self.matrix[row, col] = value

```

```

def get_rows(self) -> int:

```

```

    return self.rows

```

```

def get_cols(self) -> int:

```

```

    return self.cols

```

```

def pprint(self):

```

```

    print(self.matrix)

```

```

# функція для отримання визначника матриці

```

```

def determinant(self) -> float:

```

```

    if self.get_rows() != self.get_cols():

```

```

        raise Exception('Matrix is not square, impossible to calculate determinant')

```

```

    return np.linalg.det(self.matrix)

```

```

# функція для отримання знаку визначника матриці та натурального
логарифма визначника

```

```

def slog_determinant(self) -> tuple[float, float]:

```

```
sign, log_det = np.linalg.slogdet(self.matrix)
return sign, log_det
```

послідовний алгоритм пошуку оберненої матриці

```
def inverse_matrix(self):
```

```
    # перевірка можливості обернення матриці
```

```
    if self.get_rows() != self.get_cols():
```

```
        raise Exception('Matrix is not square, impossible to calculate inverse matrix')
```

```
    det_sign, log_det = self.slog_determinant()
```

```
    if np.round(np.exp(log_det)) == 0:
```

```
        raise Exception('Matrix determinant is 0, impossible to calculate inverse
matrix')
```

```
    inverse_matrix = Matrix(self.rows, self.cols)
```

```
    start = time.time()
```

```
    for row in range(self.rows):
```

```
        for col in range(self.cols):
```

```
            # алгебраїчне доповнення елементу matrix[row][col]
```

```
            minor_sign, log_minor = self.get_matrix_minor(row, col)
```

```
            value = np.exp(log_minor - log_det) * minor_sign * det_sign
```

```
            # якщо сума індексів непарна - змінюємо знак
```

```
            if (row + col) % 2 != 0:
```

```
                value = -value
```

```
    # транспоновано встановлюємо значення оберненої матриці
```

```
inverse_matrix.set_value(col, row, value)
```

```
end = time.time()
```

```
print(f'Sequential inversion time: {end - start}')
```

```
return inverse_matrix
```

```
# отримання мінора матриці - визначник основної матриці, у якій видалені
# задані рядок та стовпець
```

```
# результат повертається як знак визначника та значення натурального
# логарифма визначника
```

```
def get_matrix_minor(self, row_to_delete: int, col_to_delete: int) -> tuple[float,
float]:
```

```
# Отримання матриці без заданого рядка та стовпця
```

```
m = np.delete(np.delete(self.matrix, row_to_delete, axis=0), col_to_delete,
axis=1)
```

```
# обчислення визначника матриці
```

```
sign, logdet = tf.linalg.slogdet(m)
```

```
return sign.numpy(), logdet.numpy()
```

```
# алгоритм пошуку оберненої матриці із використанням пулу процесів
```

```
def inverse_matrix_process_pool(self, processes_amount: int):
```

```
# перевірка можливості обернення матриці
```

```
if self.get_rows() != self.get_cols():
```

```
    raise Exception('Matrix is not square, impossible to calculate inverse matrix')
```

```
det_sign, log_det = self.slog_determinant()
```

```
if np.round(np.exp(log_det)) == 0:
```



```
raise Exception('Matrix determinant is 0, impossible to calculate inverse
matrix')
```

```
# створення пулу процесів та списку для зберігання результатів роботи
функцій
```

```
pool = Pool(processes=processes_amount)
```

```
futures = []
```

```
start = time.time()
```

```
# додавання задач у пул процесів
```

```
for row in range(self.rows):
```

```
    futures.append(pool.apply_async(self._inverse_matrix_worker,      args=(row,
det_sign, log_det)))
```

```
pool.close()
```

```
# очікування кінця виконання задач у пулі процесів
```

```
for future in futures:
```

```
    future.wait()
```

```
# об'єднання результатів виконання задач в одну матрицю
```

```
inverse_matrix = np.column_stack([future.get() for future in futures])
```

```
end = time.time()
```

```
print(f'Process pool inversion time: {end - start}')
```

```
return Matrix(self.rows, self.cols, inverse_matrix)
```

```
# задача для процесів пошуку оберненої матриці
```

```

def _inverse_matrix_worker(self, row: int, det_sign: float, log_det: float) ->
np.ndarray:
    result = np.zeros((self.rows, 1), dtype=np.float32)

    for col in range(self.cols):
        # алгебраїчне доповнення елементу matrix[row][col]
        minor_sign, log_minor = self.get_matrix_minor(row, col)

        value = np.exp(log_minor - log_det) * minor_sign * det_sign

        # якщо сума індексів непарна - міняємо знак
        if (row + col) % 2 != 0:
            value = -value

        # транспоновано встановлюємо значення оберненої матриці
        result[col, 0] = value

    return result

def __eq__(self, other) -> bool:
    return np.all(self.matrix == other.matrix)

```

Файл main.py

```

from matrix import Matrix
import numpy as np
import tensorflow as tf
from math import exp

```

```
def single_sequential_test():
    size = 5
    m = Matrix(size, size)
    m.generate_random_matrix(-100, 100)
    print('\nMatrix:')
    m.pprint()

    inverse = m.inverse_matrix()
    print('\nInverse matrix:')
    inverse.pprint()

    print('\nInitial matrix and inverse matrix multiplication:')
    print(np.round(np.matmul(inverse.matrix, m.matrix)))

def single_parallel_test():
    size = 5
    m = Matrix(size, size)
    m.generate_random_matrix(-100, 100)
    print('\nMatrix:')
    m.pprint()

    inverse = m.inverse_matrix_process_pool(2)
    print('\nInverse matrix:')
    inverse.pprint()

    print('\nInitial matrix and parallel inverse matrix multiplication:')
    print(np.round(np.matmul(inverse.matrix, m.matrix)))
```

```

def speed_test():
    sizes = [100, 150, 200, 250, 300, 350, 400, 450]
    processes = [2, 4, 6, 8, 12, 16]

    for size in sizes:
        m = Matrix(size, size)
        m.generate_random_matrix(-100, 100)

        print(f'\n-----Matrix {size}x{size} -----\n')
        inverse = m.inverse_matrix()

        for thread_amount in processes:
            print(f'\nProcesses: {thread_amount}')
            pool_inverse = m.inverse_matrix_process_pool(thread_amount)

def main():
    # single_sequential_test()
    # single_parallel_test()
    speed_test()

if __name__ == '__main__':
    main()

```